

A Drift-Aware MLOps Framework for Multi-Model Serving: A Network Intrusion Detection Case Study

Hoang Tran-Nguyen-Viet¹, Thai Bui-Ngoc¹, Tuan Le-Anh¹, Bao Nguyen-Van¹,
and Quan Le-Trung¹

Faculty of Computer Networks and Communications,
University of Information Technology, VNU-HCM,
Ho Chi Minh City, Vietnam
{23520541,23521412}@gm.uit.edu.vn, {tuanla,baonv,quanlt}@uit.edu.vn

Abstract. Distribution alerts and predictive quality answer different monitoring questions. We quantify their disagreement in a controlled NIDS study and use it to motivate a version-associated MLOps lifecycle design. A fixed BENIGN/DDoS XGBoost model is evaluated across disjoint pools, 120 windows, two sizes, and four alert thresholds. Changing attack prevalence produces strong distribution alerts while macro-F1 remains near one. Replacing one quarter of attack samples with held-out families reduces mean attack recall to 0.7533 for 1,000-sample windows, although a BH-adjusted KS monitor alerts in only one of five windows at a drift-share threshold of 0.30. A four-window service replay supplies separate integration evidence. The prototype’s Native Registry links model versions, references, and monitoring reports; the proposed lifecycle contract separates these records from labeled quality evidence and reviewed update decisions. The contribution is the controlled study and its design implications, rather than a new detector, registry technology, or demonstrated adaptation policy.

Keywords: MLOps · Data Drift · Multi-model Serving · Network Intrusion Detection

1 Introduction

Changing traffic and attack families challenge NIDS generalization [15]; labels for measuring quality may arrive late. Shift detection and harmfulness are already distinct research questions [1]. We ask: *under which constructed changes do distribution alerts agree or disagree with lower predictive quality?* Here degradation means lower performance on changed composition, not deterioration over time.

Our principal evaluated contribution compares alerts and labeled metrics with a fixed classifier and reference. It motivates a version-associated lifecycle contract separating distribution evidence, quality evidence, and reviewed actions.

Native Registry supplies the prototype foundation; a service replay supplies integration evidence. Multi-model serving is architectural scope, whereas this study evaluates monitoring of one classifier.

2 Related Work

MLOps and technical-debt research address lifecycle components and dependencies [4,5]. TFX, MLflow, and Clipper supply pipeline, lifecycle, and serving foundations [6,7,8]; integrating these responsibilities is not itself our novelty.

Rabanser et al. distinguish shift detection from harmfulness [1]; we quantify their disagreement for controlled NIDS mixtures. Sridhar et al. address production governance and prediction provenance [2]; we apply that concern to model/reference/window/report associations. Leest et al. organize contextual information for interpreting monitoring [3]; version and window identities supply a limited instance of such context. These precedents establish the principles our prototype concretizes.

HarmonE monitors accuracy, energy, and distribution shifts to guide adaptation [9]. We evaluate disagreement before an update decision. Concept-drift analyses [10,11] and continual-learning NIDS [12] address changing concepts or updates; our classifier stays fixed. Our addition is the controlled quantification and lifecycle interpretation, not a new distinction between shift and harm.

3 Framework and Monitoring Contract

Implemented foundation. The Django control plane stores product state in PostgreSQL: project ownership, model version UUIDs, and artifact records. FastAPI resolves serving versions; Redpanda and a consumer persist inference events. Monitors link versions to reference assets; run UUIDs identify status, scores, and report URIs. MLflow, storage, and execution engines act as adapters.

Defined and evaluated scope. The contract requires evidence to retain tenant, version, reference, window, and report identities. Reference assets have UUIDs and optional checksums; snapshot/window links are optional. The replay path reads the latest version-scoped records, so these fields do not guarantee frozen membership or immutable reference bytes. Offline manifests fix membership; a separate audit reconciles pilot inputs with stored records. The proposed review and promotion conditions below remain unevaluated as lifecycle policies.

Distribution evidence. For tenant u and model version i , let $\hat{y} = f_{u,i}(x)$, with reference $D_{\text{ref}}^{u,i}$ and window $D_{\text{cur}}^{u,i}$. In the offline study, two-sample tests yield p_j for d numeric features. Benjamini–Hochberg (BH) [16] produces adjusted p-values q_j within each window:

$$I_j = \mathbf{1}[q_j < 0.05], \quad \rho = \frac{1}{d} \sum_{j=1}^d I_j, \quad A = \mathbf{1}[\rho \geq \tau_D]. \quad (1)$$

The flagged-feature fraction p depends on the rule, feature set, and sample size; it is neither shift severity, affected-request share, nor failure probability. Marginal tests can miss dependence changes. BH here does not guarantee dataset-level false-alert control under dependent, tied features.

Proposed review semantics. An alert requests inspection, not replacement. Labeled macro-F1, attack recall, and benign false-positive rate (FPR) provide separate evidence: low quality warrants review even when $A = 0$. Actions should reference their supporting report and quality window. Retraining requires representative labels; promotion also requires held-out quality, operational gates, and human review. These conditions are not certified enforcement.

4 Controlled NIDS Experiment

4.1 Provenance and Disjoint Pools

CICIDS2017 provides labeled network flows [14]. We use a team-subsampled derivative of *CICIDS2017: Cleaned & Preprocessed* dataset, followed by the team’s *MLOps NIDS Dataset*.¹ Local extraction caps labels, reserves test rows, and shuffles the remainder. Its `reference_data.csv` contains 209,865 rows: 100,000 BENIGN, 50,000 DDoS, 50,000 PortScan, 8,150 BruteForce, and 1,715 WebAttacks, with 52 numeric features.

Upstream preprocessing combines data, cleans values, groups labels, and selects features before our split. Time/capture identifiers are absent. Potential upstream leakage and complete byte-level lineage from the original release remain unverified in this study.

We hash ordered numeric feature vectors, excluding labels and normalizing signed zero. Twenty rows in ten feature-identical, conflicting-label groups and one further duplicate are removed; there are no nonfinite rows. The remaining 209,844 rows are stratified into train/development/reference/evaluation pools at 50/10/20/20% with split seed 260926. All six pool pairs have zero exact feature-vector overlap, including benign rows. This does not exclude session-level or correlated-flow dependence.

4.2 Fixed Model and Window Construction

One binary XGBoost classifier [13] is fitted on 74,994 BENIGN/DDoS rows from the train pool. PortScan, BruteForce, and WebAttacks are deliberately unseen during fitting. We use 200 trees, depth 6, learning rate 0.1, full row/column sampling, the histogram tree method, model seed 42, and prediction threshold 0.5. No scaler, SMOTE, or evaluation-based tuning is used; two development windows check execution. The model stays fixed.

¹ Kaggle source: <https://www.kaggle.com/datasets/ericanacletoribeiro/cicids2017-cleaned-and-preprocessed>; team derivative: <https://www.kaggle.com/datasets/hongtrnnguyenvit/nids-reference-data>.

Table 1. Controlled scenarios; S2 changes attack composition at fixed prevalence.

Scenario	Total attack fraction	Held-out share of attack rows
S0	30%	0% (all DDoS)
S1	15%, 50%, 85%	0% (all DDoS)
S2	30%	25%, 50%, 100%

A separate 5,000-row reference contains 70% BENIGN and 30% DDoS (reference sampling seed 260927). Evaluation windows contain $n \in \{500, 1000\}$ rows sampled without replacement within each window. Table 1 specifies the scenarios. In S2, replacement attacks are divided as evenly as possible among the three held-out families. Thus S2 at 25% replacement and $n = 1000$ contains 700 benign, 225 DDoS, and 75 held-out-family rows. Replacement percentage is a fraction of *attack rows*, not of the entire window.

Seeds are 42, 123, 456, 789, and 1024 per setting; S0 adds 2000–2024, giving 30 windows per size and 120 overall. Windows reuse the evaluation pool with fixed class counts. SD describes sampling with one model/reference, not independent captures or fits.

4.3 Detection, Metrics, and Reproducibility

Two-sided asymptotic Kolmogorov–Smirnov tests cover all 52 features (6,240 tests), followed by Eq. 1. We compare thresholds 0.10, 0.30, 0.50, and 0.70, retaining 0.30 as the principal setting and uncorrected tests as a comparison. Each window yields macro-F1, attack recall, benign FPR, ROC-AUC, family recall, and KS statistics. No threshold is selected retrospectively.

The retained local run `run_20260926` includes `dataset_manifest.json` (input SHA-256, ordered schema, exclusions), `model_manifest.json` (model SHA-256), compressed split/window memberships, and `reference_manifest.csv`. Config/environment files preserve seeds, hyperparameters, and versions, including XGBoost 3.2.0, SciPy 1.17.1, and scikit-learn 1.8.0. Per-window metrics and feature tests feed the summary tables; `build_result_assets.py` formats the main table. The separate `verify_results.py` reloads the model, recomputes predictions and all KS tests, and checks BH against SciPy; its retained verification record reports all checks passed. This supports numerical checking with the frozen local input, not independent reconstruction from Kaggle. Public artifact access and versioned input acquisition are not established; upstream byte-level lineage remains unverified.

5 Results

5.1 Distribution Drift and Predictive Quality

Table 2 reports the five principal windows per scenario at $n = 1000$. S1 changes prevalence while retaining the learned attack family. Despite drift shares of

Table 2. Fixed-model results for 1,000-row windows: mean $\pm$ sample SD over five seeds. Alerts use $\tau_D = 0.30$; S0 alert count uses all 30 stable windows.

Scenario	Drift share	Macro-F1	Attack recall	Alerts
S0	0.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0/30
S1 (15%)	0.742 $\pm$ 0.059	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	5/5
S1 (50%)	0.850 $\pm$ 0.077	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	5/5
S1 (85%)	0.981 $\pm$ 0.000	0.999 $\pm$ 0.001	1.000 $\pm$ 0.001	5/5
S2 (25%)	0.165 $\pm$ 0.202	0.905 $\pm$ 0.002	0.753 $\pm$ 0.004	1/5
S2 (50%)	0.865 $\pm$ 0.073	0.789 $\pm$ 0.003	0.508 $\pm$ 0.005	5/5
S2 (100%)	0.981 $\pm$ 0.000	0.422 $\pm$ 0.007	0.010 $\pm$ 0.007	5/5

0.7423–0.9808, macro-F1 remains at least 0.9992. These alerts correctly reflect changed input composition; unchanged predictive quality does not make them false distribution alerts.

S2 instead tests held-out attack families. At 25% replacement, attack recall falls to 0.7533, while the principal detector alerts in only 1/5 windows. The same 1/5 count occurs at $n = 500$, where mean recall is 0.7520. These held-out attacks occupy only 7.5% of the full window: substantial recall loss within the attack class can coexist with infrequent dataset alerts. At full replacement, recall is 0.0100 for $n = 1000$: PortScan and WebAttacks recall are zero and Brute-Force recall averages 0.0300. This is a limitation of the deliberately restricted BENIGN/DDoS baseline, not an evaluation of a model trained on all families.

The S2 trend must be interpreted as a mixture effect: for replacement fraction r , aggregate attack recall equals $(1 - r)R_{\text{DDoS}} + rR_{\text{held-out}}$. Post-hoc inspection of the full evaluation pool finds 9,999/10,000 DDoS, 4/9,998 PortScan, 60/1,630 BruteForce, and 0/344 WebAttacks predicted as attacks. Increasing poorly recognized families therefore explains the decline; it does not demonstrate loss of DDoS knowledge over time. This inspection reuses the evaluation pool and is not an independent test.

S1 at 85% prevalence and S2 at full replacement both flag approximately 98% of features, yet their macro-F1 values differ greatly. Drift share thus cannot rank predictive harm in these cases. Across all 30 S0 windows at $n = 1000$, mean macro-F1 is 0.999881, recall 0.999889, and FPR 0.000095; perfect values in the five-window table do not hold for every stable draw. S1 at 85% prevalence has mean benign FPR 0.001333 in the principal windows; the other table rows have zero observed FPR.

5.2 Threshold and Window-Size Sensitivity

Table 3 shows alert counts, with 30 windows for each S0 size and five for each shifted setting. All 60 S0 windows have zero BH alerts across the tested thresholds; this finite, conditional observation is not a zero-false-alarm guarantee. At 25% replacement and $n = 1000$, lowering τ_D from 0.30 to 0.10 increases alerts

Table 3. BH-corrected alert counts across drift-share thresholds. S1 labels give total attack prevalence; S2 labels give the fraction of attack rows replaced by held-out families. S0 uses 30 windows per size; other scenarios use five.

Scenario	$n = 500$				$n = 1000$			
	.10	.30	.50	.70	.10	.30	.50	.70
S0	0/30	0/30	0/30	0/30	0/30	0/30	0/30	0/30
S1 (15%)	5/5	5/5	5/5	3/5	5/5	5/5	5/5	3/5
S1 (50%)	5/5	5/5	5/5	4/5	5/5	5/5	5/5	5/5
S1 (85%)	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5
S2 (25%)	1/5	1/5	0/5	0/5	2/5	1/5	1/5	0/5
S2 (50%)	5/5	5/5	4/5	1/5	5/5	5/5	5/5	5/5
S2 (100%)	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5

from 1/5 to 2/5, but does not eliminate missed shifts. At 50% replacement and $\tau_D = 0.70$, alerts increase from 1/5 for $n = 500$ to 5/5 for $n = 1000$.

Correction also changes sensitivity. Without BH, S0 at $n = 1000$ and $\tau_D = 0.10$ produces 3/30 alerts instead of 0/30. Conversely, S2 at 25% replacement and $\tau_D = 0.30$ produces 5/5 alerts instead of 1/5. This trade-off should be reported rather than choosing a test after observing its outcomes. Window size affects statistical power; these non-temporal windows do not measure detection delay.

5.3 Controlled Service Replay

Four seed-42, 1,000-row windows used identical model bytes and the 5,000-row reference under separate version UUIDs, each initially without production records. S1 used 85% DDoS. Labels were withheld until scoring. Each export had 1,000 unique prediction IDs and request IDs; exact 52-feature-vector matching established one record per frozen input. The browser did not send the handoff row ID, so reconciliation used vectors, not retry identity.

S0’s consumer stored 700 class-0 predictions as empty strings. Scoring used all 1,000 UI predictions cross-checked against database records, allowing only that empty-string/zero discrepancy. S1/S2 used post-fix database predictions without blanks. All four prediction sets matched the offline artifact. S1 incurred 42 HTTP 502/503 failures during a worker interruption: recovery submitted those inputs plus 824 unsent inputs, yielding 1,000 records from 1,042 attempts. The audit establishes final coverage, not retry idempotency or unique failure causes.

The service source pins Evidently 0.4.15 and uses `DataDriftPreset` without feature-test overrides. With 5,000 reference rows, its defaults are normalized Wasserstein distance for numeric columns with more than five distinct values, Jensen–Shannon distance otherwise, and feature threshold 0.1, without BH. This source configuration is not an archived runtime dependency inventory. Reports used dataset threshold 0.60 with all 52 features present. Table 4 is separate integration evidence, not replication of offline KS/BH.

Table 4. Single-window live replay per setting; dataset threshold 0.60.

Window	Flagged features	Alert	Attack recall
S0	1/52	No	1.0000
S1 (85%)	43/52	Yes	1.0000
S2 (50%)	22/52	No	0.5067
S2 (100%)	29/52	No	0.0100

6 Discussion and Limitations

S1 produces alerts without material measured harm. Conversely, offline S2-r25 remains unflagged in four of five windows at $\tau_D = 0.30$, while the reported live S2-r50/r100 windows remain unflagged at the service threshold of 0.60 despite low attack recall. These counterexamples motivate separate version-associated evidence and review, not a proven improvement in update decisions. Native Registry concretizes this contract rather than introducing registry technology.

Upstream selection, missing time identifiers, correlated flows, and separability of familiar classes limit generalization. Exact deduplication does not resolve these issues. Five shifted windows per setting, fixed class counts, one model/reference, and tied-feature tests limit inference. S2 changes composition, not demonstrated $P(Y | X)$ over time. Natural drift and adaptation need separate evaluation.

The pilot does not verify frozen associations under concurrent changes or failure recovery. Its HTTP errors and the earlier 89-second Locust replay do not establish availability or scalable serving. Multi-model load, lifecycle ablations, and promotion/rollback benefits remain unevaluated. Ownership checks do not sandbox uploaded code: restricted artifact formats, isolated identities, network/resource limits, and adversarial testing are needed for untrusted workloads; such isolation is untested.

7 Conclusion

The controlled NIDS study quantifies disagreement between distribution alerts and labeled predictive quality across traffic mixtures, window sizes, and thresholds. Its strongest evidence supports separate interpretation of these signals. The version-associated lifecycle contract and Native Registry prototype make that separation concrete; the service replay adds integration evidence, while update effectiveness and broader operational guarantees remain open.

Acknowledgments. This research was supported by the VNUHCM-University of Information Technology’s Scientific Research Fund.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Rabanser, S., Günnemann, S., Lipton, Z.: Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift. In: *Advances in Neural Information Processing Systems*, vol. 32 (2019)
2. Sridhar, V., Subramanian, S., Arteaga, D., Sundararaman, S., Roselli, D., Talagala, N.: Model Governance: Reducing the Anarchy of Production ML. In: *USENIX ATC*, pp. 351–358 (2018)
3. Leest, J., Raibulet, C., Lago, P., Gerostathopoulos, I.: From Tea Leaves to System Maps: A Survey and Framework on Context-Aware Machine Learning Monitoring. *IEEE Transactions on Software Engineering* **51**(12), 3218–3246 (2025). <https://doi.org/10.1109/TSE.2025.3602520>
4. Kreuzberger, D., Kühn, N., Hirschl, S.: Machine Learning Operations (MLOps): Overview, Definition, and Architecture. *IEEE Access* **11**, 31866–31879 (2023). <https://doi.org/10.1109/ACCESS.2023.3262138>
5. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., Dennison, D.: Hidden Technical Debt in Machine Learning Systems. In: *Advances in Neural Information Processing Systems*, vol. 28 (2015)
6. Baylor, D., et al.: TFX: A TensorFlow-Based Production-Scale Machine Learning Platform. In: *KDD*, pp. 1387–1395 (2017). <https://doi.org/10.1145/3097983.3098021>
7. Zaharia, M., et al.: Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin* **41**(4), 39–45 (2018)
8. Crankshaw, D., Wang, X., Zhou, G., Franklin, M.J., Gonzalez, J.E., Stoica, I.: Clipper: A Low-Latency Online Prediction Serving System. In: *USENIX NSDI*, pp. 613–627 (2017)
9. Bhatt, H., Biswas, S., Rakhunathan, S., Vaidhyanathan, K.: HarmonE: A Self-Adaptive Approach to Architecting Sustainable MLOps. arXiv:2505.13693v1 (2025)
10. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A Survey on Concept Drift Adaptation. *ACM Computing Surveys* **46**(4), article 44, 1–37 (2014). <https://doi.org/10.1145/2523813>
11. Webb, G.I., Hyde, R., Cao, H., Nguyen, H.-L., Petitjean, F.: Characterizing Concept Drift. *Data Mining and Knowledge Discovery* **30**(4), 964–994 (2016). <https://doi.org/10.1007/s10618-015-0448-4>
12. Zhang, X., Zhao, R., Jiang, Z., Chen, H., Ding, Y., Ngai, E.C.H., Yang, S.-H.: Continual Learning with Strategic Selection and Forgetting for Network Intrusion Detection. arXiv:2412.16264v4 (2025)
13. Chen, T., Guestrin, C.: XGBoost: A Scalable Tree Boosting System. In: *KDD*, pp. 785–794 (2016). <https://doi.org/10.1145/2939672.2939785>
14. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In: *ICISSP*, pp. 108–116 (2018). <https://doi.org/10.5220/0006639801080116>
15. Sommer, R., Paxson, V.: Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. In: *IEEE Symposium on Security and Privacy*, pp. 305–316 (2010). <https://doi.org/10.1109/SP.2010.25>
16. Benjamini, Y., Hochberg, Y.: Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B* **57**(1), 289–300 (1995). <https://doi.org/10.1111/j.2517-6161.1995.tb02031.x>